

Deadlocks

Deadlocks

Deadlock : A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

➤ Example

- System has 2 disk drives.
- P_1 and P_2 each hold one disk drive and each needs another one.

➤ Example

- semaphores A and B, initialized to 1

P_0	P_1
wait (A);	wait(B);
wait (B);	wait(A);

Deadlocks...

- A process is *deadlocked* if it is waiting for an event that will never occur.
 - Typically, more than one process will be involved in a deadlock.
- A process is *indefinitely postponed* if it is delayed repeatedly over a long period of time while the attention of the system is given to other processes,
 - i.e. the process is ready to proceed but never gets the CPU.

System Model

- Resource types
 - $R_1, R_2, \dots, R_n$
- Each resource type R_i has W_i instances
- Assume reusable resources
 - request $\rightarrow$ use $\rightarrow$ release
- Request & release are system calls.(open/close file, alloc/free memory location etc). It can be accomplished with wait and signal operations on semaphore.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 and so on. ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 . i.e A circular chain of processes exists in which each process waits for one or more resources held by the next process in the chain.

Resource-Allocation Graph

- A set of vertices V and a set of edges E .

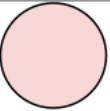
V is partitioned into two types:

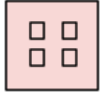
- $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
- $R = \{R_1, R_2, \dots, R_m\}$, the multi-set consisting of all resource types in the system.

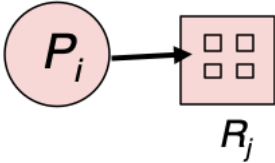
Request edge – directed edge $P_i \rightarrow R_j$

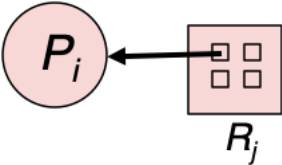
Assignment edge – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph...

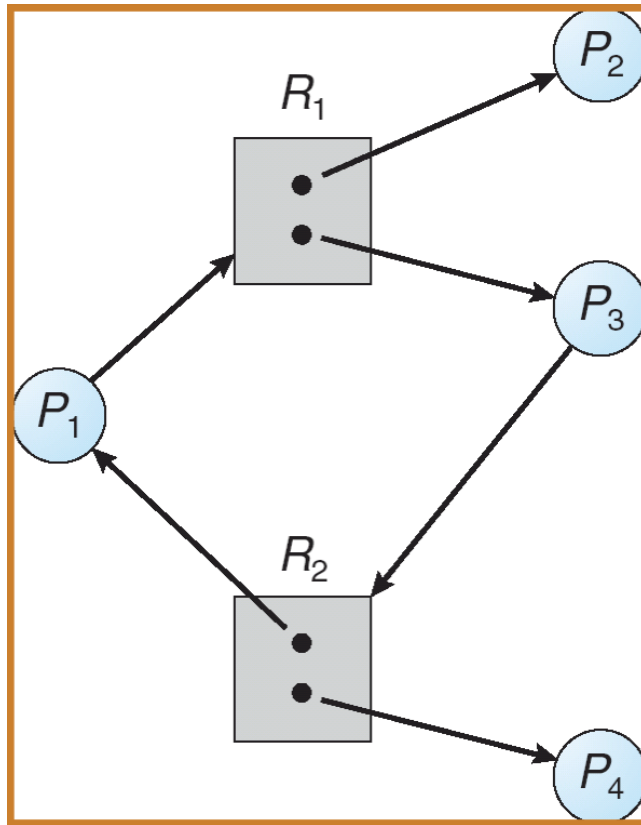
■ Process 

■ Resource Type with 4 instances 

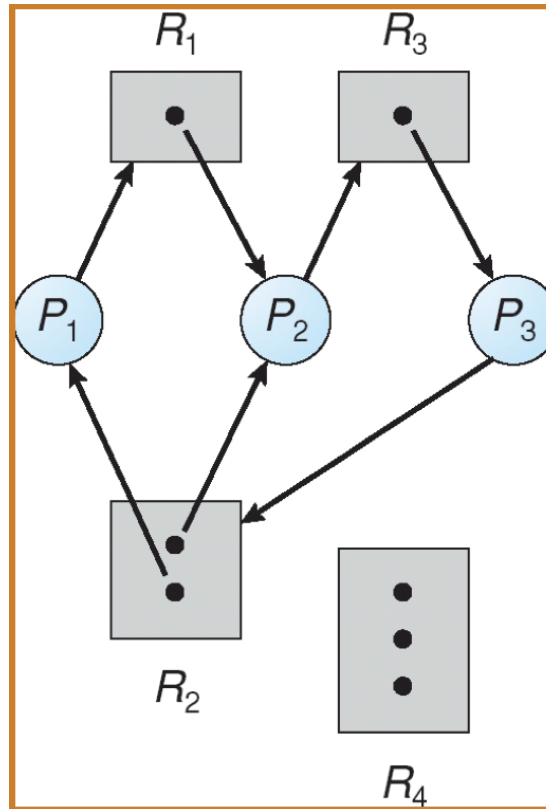
■ P_i requests an instance of R_j 

■ P_i is holding an instance of R_j 

Graph With A Cycle But No Deadlock



Resource Allocation Graph With A Deadlock



Basic Facts

If graph contains no cycles

- no deadlock.

If graph contains a cycle

- if only one instance per resource type, then deadlock.
- if several instances per resource type, possibility of deadlock.

Methods for handling deadlocks

- Ensure that the system will never enter a deadlock state. Use a Protocol for **prevent** or **avoid** deadlocks.
- Allow the system to potentially enter a deadlock state, **detect** it and then **recover**.
- Ignore the problem and pretend that deadlocks never occur in the system;
 - Used by many operating systems, e.g. UNIX

Deadlock Management

- **Prevention**

- Design the system in such a way that deadlocks can never occur. (by constraining how requests for resources can be made in the system)
- Provides set of methods for ensuring that at least one of the necessary conditions cannot hold.

- **Avoidance**

- The system dynamically considers every request and decides whether it is safe to grant it at this point.
- The system requires additional appropriate information regarding the overall potential use of each resource for each process.

Deadlock Management...

- **Detection**

- Allow possibility of deadlock, determine if deadlock has occurred and which processes and resources are involved.

- **Recovery**

- After detection, clear the problem, allow processes to complete and resources to be reused. May involve destroying and restarting processes.

Deadlock Prevention

- If any one of the conditions for deadlock (with reusable resources) is denied, deadlock is impossible.
 - **Mutual Exclusion**
 - no-issue for sharable resources
 - cannot deny this for non-sharable resources.
 - **Hold and Wait** - guarantee that when a process requests a resource, it does not hold other resources.
 - Force each process to acquire all the required resources at once (before execution).
 - The process will make a new request for resources after releasing the current set of resources.
 - Low resource utilization.
 - Starvation is possible.

Deadlock Prevention...

■ No Preemption

- Preempt the resources from process.
- Preempted resources are added to the list of resources for which the process is waiting.
- Process will be restarted only when it can regain its old resources as well as the new ones that it is requesting.
- When a process request for some resources, first check if its available, allocate if so. Otherwise check if its allocated to the waiting process. If so preempt the resources of a waiting process and allocate to the requesting process. If both don't hold, the requesting process is made to wait.

Deadlock Prevention...

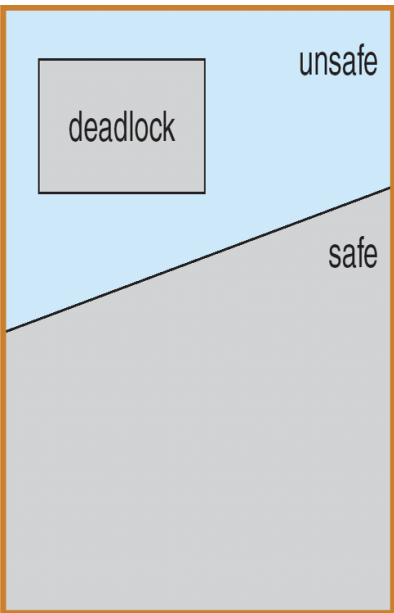
■ Circular Wait

- Impose a total ordering of all resource types.
- Require that processes request resources in increasing order of enumeration; if a resource of type N is held, process can only request resources of types $> N$.
- Let $R = \{ R_1, R_2, \dots, R_m \}$ be resource types. Assign each resource type a unique natural number, which allows to compare between two resources and to determine whether one proceeds another in ordering. $F: R \rightarrow \mathbb{N}$.
- Initially process can request any number of instances of a resource type.
- After that process can request a resource R_j only if $F(R_j) > F(R_i)$.
- In this way circular wait is prevented.
- i.e $F(R_0) < F(R_1) < F(R_2) \dots < F(R_n) < F(R_0)$ is impossible.

Deadlock Avoidance

- Requires that the system has some additional a priori information available.
- Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes.

Safe State



- System is in *safe* state if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the system such that for each P_j , the resources that P_j can still request can be satisfied by currently available resources + resources held by all the P_i , with $i < j$.
- That is:
 - If P_j resource are not immediately available, then P_j can wait until all P_i have finished.
 - When P_i is finished, P_j can obtain needed resources, execute, return allocated resources, and terminate.
 - When P_j terminates, P_{j+1} can obtain its needed resources, and so on.

Safe State...

- System is in a safe state $\Rightarrow$ no deadlocks
- If system is deadlocked $\Rightarrow$ state is unsafe
- System is in unsafe state $\Rightarrow$ possibility of deadlock.
 - OS cannot prevent processes from requesting resources in a sequence that leads to deadlock
- Deadlock Avoidance
 - Ensures system will never enter an unsafe state.
 - Thereby avoids the possibility of deadlock.

Deadlock Avoidance - Concepts

Key Ideas:

- Initially the system is in a safe state
- Whenever a process requests an available resource, system will allocate resource immediately only if the resulting state is still safe!
- Otherwise, requesting process must wait.

Why does this work?

- By induction, all reachable states are safe states
- By definition, all safe states are not deadlocked

Avoidance algorithms

Single instance of a resource type

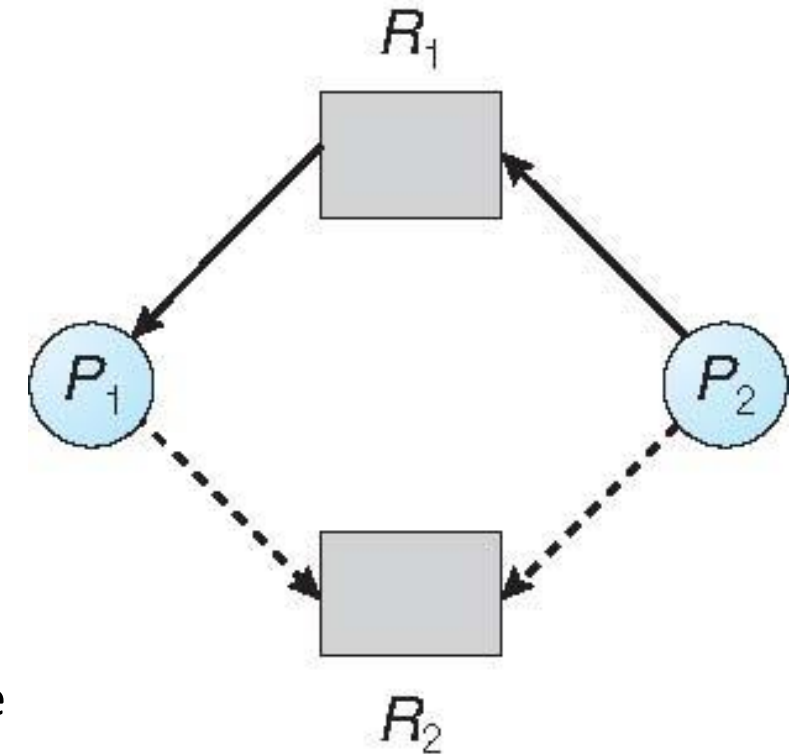
- Use a resource-allocation graph

Multiple instances of a resource type

- Use the banker's algorithm

Resource-Allocation Graph

- Claim edge: $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j ; represented by a dashed line.
- Claim edge converts to request edge when a process requests a resource.
- When a resource is released by a process, assignment edge reconverts to claim edge.
- Resources must be claimed a priori in the system.
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph.
- If request assignment does not result in the formation of a cycle in the resource allocation graph - safe state, else unsafe state.



Banker's Algorithm

- Used for multiple instances of each resource type.
- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need.
- This number should not exceed the number of resources in a system.
- When a process requests a resource, the OS checks if the allocation leaves the system in a safe state. If so, the OS allocates the resources, otherwise the process must wait.
- When a process gets all its resources, it must return them in a finite amount of time.

Data Structures for the Banker's Algorithm

Let n = number of processes and m = number of resource types.

- **Available:** Vector of length m . If $Available[j] = k$, there are k instances of resource type R_j available.
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j .
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$, then process P_i is currently allocated k instances of resource type R_j .
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then process P_i may need k more instances of resource type R_j to complete its task.

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm

1. Let *Work* and *Finish* be vectors of length m (number of resources) and n (number of processes), respectively. Initialize

Work := Available

Finish[i] := *false* for $i = 0, 1, 2, \dots, n-1$.

2. Find an i (i.e. process P_i) such that both:

(a) *Finish*[i] = *false*

(b) $Need_i \leq Work$

If no such i exists, go to step 4.

3. *Work* := *Work* + *Allocation* $_i$

Finish[i] := *true*

go to step 2

4. If *Finish*[i] = *true* for all i , then the system is in a safe state.

Example

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A\ B\ C$	$A\ B\ C$	$A\ B\ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example...

The content of the matrix *Need* is defined to be *Max – Allocation*

	<u>Need</u>
	<i>A B C</i>
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

The system is in a safe state since the sequence $\langle P_1, P_3, P_0, P_2, P_4 \rangle$ satisfies safety criteria

Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If $Request_i[j] = k$, then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise, P_i must wait since resources are not available.
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available := Available - Request_i;$

$Allocation_i := Allocation_i + Request_i;$

$Need_i := Need_i - Request_i;$

4. If safe $\Rightarrow$ resources are allocated to P_i .
5. If unsafe $\Rightarrow$ P_i must wait and the old resource-allocation state is restored.

Example: P_1 Request (1,0,2)

Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Executing safety algorithm shows that sequence $\langle P_1, P_3, P_0, P_2, P_4 \rangle$ satisfies safety requirement

Can request for (3,3,0) by P_4 be granted?

Can request for (0,2,0) by P_0 be granted?

Deadlock Detection

- Allow system to enter deadlock state
- Provide an Algorithm that examines the state of the system to determine whether a deadlock has occurred (deadlock detection)
- Note: Detection scheme requires overhead that includes the runtime costs of maintaining the necessary information and executing the detection algorithm.

Single Instance of Each Resource Type

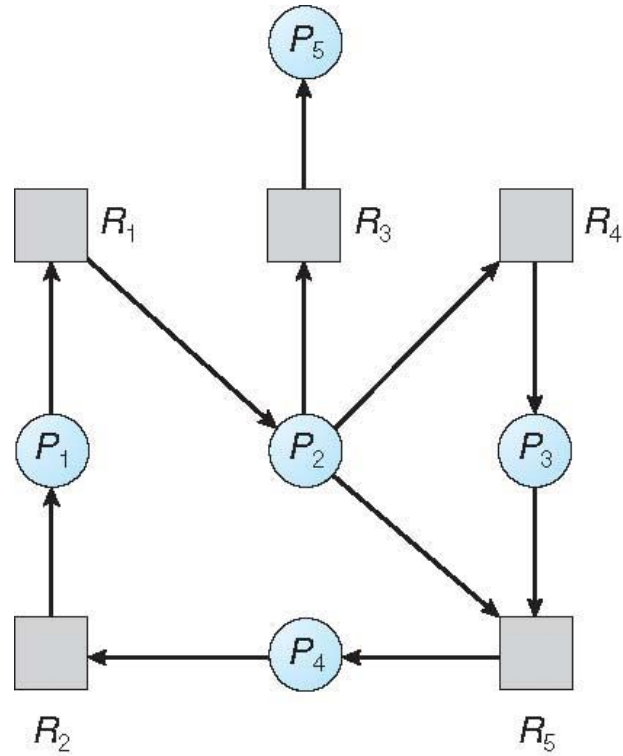
Maintain *wait-for* graph

- Remove the resource nodes from RA graph. So now nodes are only processes
- Remove the corresponding edges. i.e Instead of $P_i \rightarrow R_q$ & $R_q \rightarrow P_j$ for some resource R_q , use $P_i \rightarrow P_j$ if P_i is waiting for P_j .

Periodically invoke an algorithm that searches for a cycle in the graph.

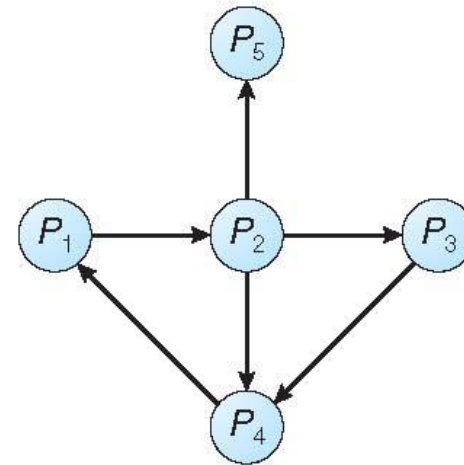
An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Example



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

Several Instances of a Resource Type

Data Structures

- **Available:** Vector of length m . If $\text{Available}[j] = k$, there are k instances of resource type R_j available.
- **Allocation:** $n \times m$ matrix. If $\text{Allocation}[i,j] = k$, then process P_i is currently allocated k instances of resource type R_j .
- **Request :** An $n \times m$ matrix indicates the current request of each process. If $\text{Request}[i,j] = k$, then process P_i is requesting k more instances of resource type R_j .

Deadlock Detection Algorithm

Step 1: Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize:

- (a) *Work* = *Available*
- (b) For $i = 1, 2, \dots, n$, if $\text{Allocation}_i \neq 0$, then
 $\text{Finish}[i] = \text{false}$; otherwise, $\text{Finish}[i] = \text{true}$

Step 2: Find an index *i* such that both:

- (a) $\text{Finish}[i] == \text{false}$
- (b) $\text{Request}_i \leq \text{Work}$

If no such *i* exists, go to **Step 4**.

Step 3: $\text{Work} := \text{Work} + \text{Allocation}_i$
 $\text{Finish}[i] := \text{true}$
 go to **Step 2**

Step 4: If $\text{Finish}[i] = \text{false}$ for some i , $1 \leq i \leq n$, then the system is in a deadlock state. Moreover, if $\text{Finish}[i] = \text{false}$, the P_i is deadlocked.

$O(m \times n^2)$

Example of Detection Algorithm

Five processes P_0 through P_4 ; three resource types
A (7 instances), B (2 instances), and C (6 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0 0 0 0	
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i

Example of Detection Algorithm...

P_2 requests an additional instance of type C

	<u>Request</u>
	$A \ B \ C$
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

State of system?

Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests

Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4

Detection-Algorithm Usage

➤ When, and how often to invoke depends on:

- How often a deadlock is likely to occur?
- How many processes/threads will be affected?

➤ How often?

- Every time a request for allocation cannot be granted immediately
- Allows us to detect set of deadlocked processes and process that “caused” deadlock. Extra overhead.
- Every hour or whenever CPU utilization drops. With arbitrary invocation there may be many cycles in the resource graph and we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

Recovery from Deadlock

➤ When the detection algorithm detects that deadlock exists either user can deal with it manually or OS automatically recover from deadlock.

Two options to break deadlock automatically

- **Abort** one or more processes
- **Preempt** some resources from one or more deadlocked processes.

Process Termination

- Abort all deadlocked processes
 - Very expensive as processes may have computed for a long time & this must be discarded and recompute it later.
- Abort one process at a time until the deadlock cycle eliminated
 - Adds up overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.
- Issues with aborting the processes
 - If the process is in mid of printing/updating file then OS must reset the values before proceeding with next request.
 - Factors to be considered while terminating the processes
 - Priority of the processes
 - How many and what types of resources are used/ required to complete?
 - Foreground process/ Background process?
 - How long the process has executed and how much longer is required to complete the task.

Resource Preemption

- To eliminate deadlocks, preempt some of resources and give these resources to other processes until the deadlock cycle is broken.
- 3 Issues need to be addressed
 - **Selecting a victim:** Which resources and which processes are to be preempted?
- Solⁿ: Select the one to minimize the cost. Parameters can be number of resources held, How long the process has computed etc.
 - **Rollback:** What should be done with the process for which resources are preempted?
- Solⁿ: Either rollback the process from safe state (overhead added to OS) or total rollback (abort the process and then restart it)
 - **Starvation:** How can we guarantee that resources will not always be preempted from the same process?
- Solⁿ: Include the number of rollbacks in the cost factor while selecting the victim.